

A task-specific visual token access threshold

What visual token pruning removes, and when it is safe to remove it

P. S. Kedar | Supervised by Prof. Divya Saxena | Mentored by Aditya Sharma, PhD Scholar

IIT Jodhpur | Phase 3 | September 2026

Qwen stopped reading

- BTP reproduces cleanly on LLaVA-1.5-7B, LLaVA-1.5-13B and Qwen2.5-VL-7B
- But on tasks that need the model to read text inside an image, Qwen collapses
- AI2D has text and rarely needs it read word by word. It barely moves, so it is our control
- The paper reports no reading task, so nothing in its results would show this

Benchmark	Baseline	BTP
TextVQA	86.2	23.3
DocVQA	94.7	19.2
ChartQA	76.8	29.0
AI2D (control)	86.4	79.6

Qwen2.5-VL-7B, limit 500

Four explanations, all eliminated

Hypothesis	Test	Result
Selector picks badly	Diversity, attention, random at one budget	17.3 to 19.4, inside the error bars
Text patches look alike	Feature similarity at layer 4	Text 0.379, background 0.438
Too much is removed	Retention swept 100 to 12.5 per cent	90 per cent already gives the full drop
Wrong tokens kept	Ground-truth OCR oracle	0.197 oracle, 0.197 control

Four unrelated interventions, one number. The control that broke it open: at 100 per cent retention, where nothing is pruned at all, the score is still 0.178. The loss was happening after the pruning stages.

One branch, no image index

```
elif layer_index == self.end_layer:    # 23
    remain_sys_index = ...
    remain_text_index = ...
    # no remain_img_index below
    all_remained_index = torch.cat(
        (remain_sys_index, remain_text_index))
```

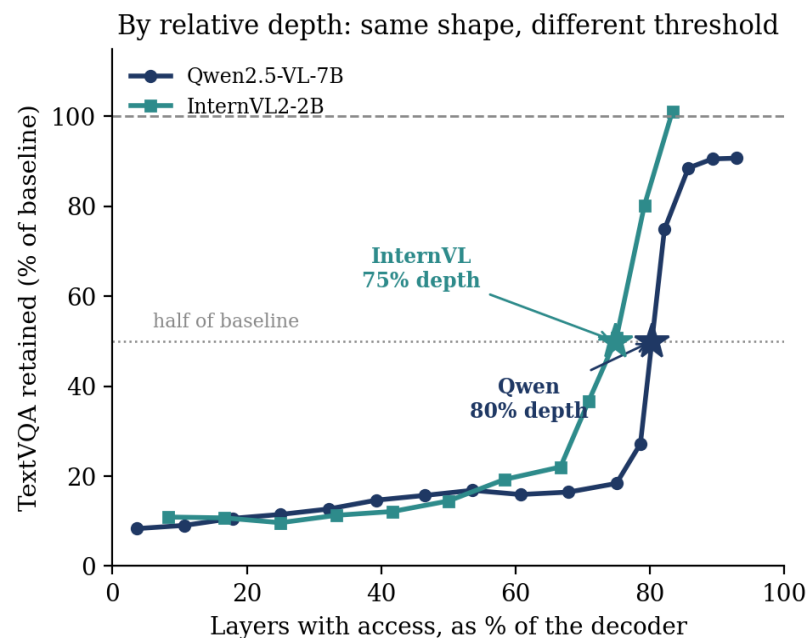
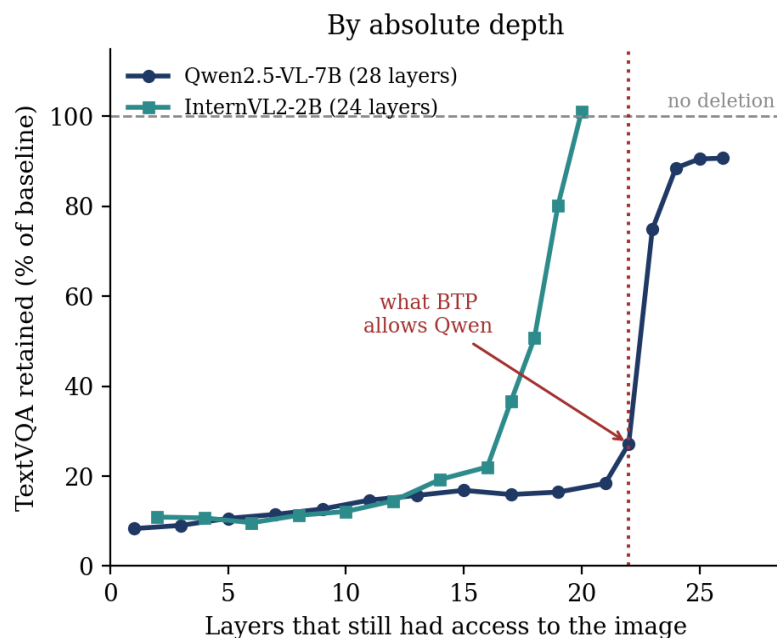
- Every image token surviving three pruning stages is removed here, unconditionally
- Appendix 7.3 specifies this for LLaVA only. For Qwen it asks for 12.5 per cent retention
- Code at commit 9682db0, file modeling_qwen2_5_vl.py, branch at line 1385

Arm	TextVQA	AI2D
No pruning	0.8565	0.8750
BTP as released	0.1725	0.8150
Deletion disabled	0.7860	0.8100

One condition changed, the 12.5 per cent pruning left intact. Job 416735, 200 samples.

THE THRESHOLD

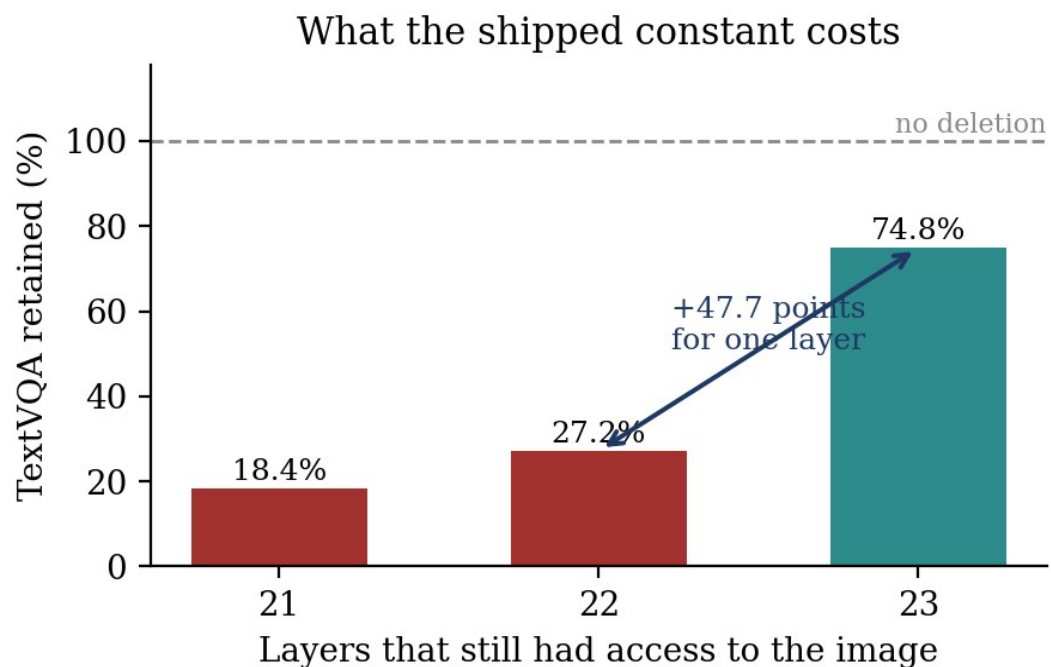
Is layer 23 special, or just early?



- We moved the deletion across the decoder, graded pruning off
- Both models sit on a floor, then climb sharply
- Threshold: the half-of-baseline crossing, interpolated
- Measured on TextVQA, so it is task-specific

Model	Layers	Threshold	Depth
InternVL2-2B	24	18.0	74.8 %
Qwen2.5-VL-7B	28	22.5	80.3 %

The released constant lands one layer short



22

layers BTP allows Qwen

22.5

measured threshold

47.7 points

TextVQA retention, one layer either side of the threshold

BTP does calibrate. Section 4.3 selects the three pruning layers from 64 samples. The depth at which complete deletion becomes safe is a different quantity, and does not appear to be calibrated.

None of this is visible on the paper's benchmarks

	Paper's suite	Reading tasks
BTP as released	96.1 %	28.4 %
BTP as specified	96.9 %	77.9 %
InternVL, no discrepancy	94.9 %	56.5 %

Same code, two verdicts.

The paper claims 96 to 98 per cent. The released code, discrepancy included, scores 96.1 on its own suite.

And on InternVL2-2B, where we found no such discrepancy, correct pruning still loses half the reading ability while the same suite reports 95 per cent.

Percentage of each model's own unpruned baseline. InternVL row is graded pruning only, with no deletion at all.

The blind spot is in the evaluation, not only in one implementation.

Two costs, and they are not the same

Benchmark	Cost of pruning	Cost of deletion	Dominant
TextVQA	5.7	57.2	deletion
DocVQA	17.9	57.6	deletion
ChartQA	31.4	16.4	pruning
AI2D (control)	7.2	0.4	pruning
GQA	2.1	2.9	both small
POPE	1.3	0.1	both small
MMBench-EN	4.7	0.3	both small

ChartQA reverses

Pruning costs twice what the deletion does there, so the discrepancy is not the dominant problem on every reading task.

An earlier version of our analysis treated deletion as the main cause throughout. These figures do not support that.

Baseline minus corrected is the pruning cost. Corrected minus released is the deletion cost. Qwen2.5-VL-7B.

Three findings, at three levels

Implementation

The released code deletes all remaining image tokens just below the threshold we measured for Qwen, while the paper's appendix asks for retention.

Commit 9682db0. Says nothing about author intent, or about Qwen as a model.

Model

Visual token access has a measurable, abrupt depth threshold. 80.3 per cent in Qwen, 74.8 in InternVL.

Two models, measured on TextVQA. Task-specific.

Evaluation

The benchmark suite did not reveal the degradation in reading, with the discrepancy present or absent.

The one finding that does not depend on the other two.

What we would not claim, and what is next

Limitations

- Two architectures. The matching shape is a pattern, not a law
- Thresholds measured on TextVQA only. ChartQA may differ
- The InternVL pruning is ours, diversity selector only, layers transplanted not calibrated
- The OCR oracle ran at one budget
- 500 sample subsets, matched across arms

Claims withdrawn

- Text has no redundancy. Our own measurement says the opposite, 0.379 against 0.438
- Text coverage predicts correctness. Vanished at a larger sample
- The effect is Qwen-specific. The sweep showed both models have a threshold
- Qwen's threshold is at 83.9 per cent. A counting error; it is 80.3

Next: thresholds on DocVQA and ChartQA, ChartQA first. Then a third architecture, then the manuscript.

Measure the depth. Test the reading.

A pruning method has to choose a depth at which visual tokens stop being needed. That depth is measurable, it differs by architecture, and the sweep is cheap: about seven evaluations, no training, no calibration set.

And include one reading benchmark. Not because reading matters more than reasoning, but because it degrades first when visual evidence is removed.

Report, results log and code:
github.com/milab-iitj-dev/btp-reproduction